

Case Study 1 - Unique Email Registration System

Java Collections Framework — Case Study

Problem Statement

A company allows users to register using their email addresses. The system must ensure that the same email address cannot be registered more than once.

Requirements Overview

Task	Collection	What Interns Build
Email Registration	HashSet	Unique email storage and validation

Requirements

- Create an email registration system.
- Store registered email addresses.
- Prevent duplicate email registrations.
- Check whether an email already exists.
- Display all registered emails.

Appropriate Collection

HashSet<String>

Justification

- Stores only unique values.
- Duplicate emails are automatically rejected.
- Fast lookup using `contains()`.

Sample Input

```
abc@gmail.com
xyz@gmail.com
abc@gmail.com
```

Expected Output

```
abc@gmail.com registered successfully.
xyz@gmail.com registered successfully.
abc@gmail.com already exists.
```

Registered Emails:

```
[abc@gmail.com, xyz@gmail.com]
```

Case Study 2 - Product Catalog Sorting

Java Collections Framework — Case Study

Problem Statement

An e-commerce application displays products. Products should be sorted differently based on system defaults and user preferences.

Requirements Overview

Sorting Type	Concept Used	Criteria
Default Sorting	Comparable	Product ID
User Preference	Comparator	Product Name
User Preference	Comparator	Product Price

Requirements

Create a Product class containing:

- Product ID
- Product Name
- Product Price

Implement:

- Default sorting by Product ID.
- Sorting by Product Name.
- Sorting by Product Price.

Appropriate Approach

Comparable

- Natural ordering.
- Product ID sorting.

Comparator

- Multiple custom sorting options.
- Name sorting.
- Price sorting.

Sample Input

```
101 Laptop 50000
103 Mouse 500
102 Keyboard 1000
```

Expected Output

Sorted by ID

```
101 Laptop 50000
102 Keyboard 1000
103 Mouse 500
```

Sorted by Name

```
102 Keyboard 1000
101 Laptop 50000
103 Mouse 500
```

Sorted by Price

```
103 Mouse 500
102 Keyboard 1000
101 Laptop 50000
```

Case Study 3 - Library Book Management System

Java Collections Framework — Case Study

Problem Statement

A library stores book records and must prevent duplicate Book IDs while allowing quick retrieval of book information.

Requirements Overview

Task	Collection	What Interns Build
Book Management	HashMap	Book storage and retrieval

Requirements

Store:

- Book ID
- Book Title

Perform:

- Add books.
- Prevent duplicate Book IDs.
- Retrieve book information using Book ID.
- Display all books.

Appropriate Collection

HashMap<Integer, String>

Justification

- Keys are unique.
- Fast retrieval using Book ID.
- Easy to display records.

Sample Input

```
101 Java Programming
102 Spring Boot
103 SQL Basics
```

Expected Output

```
All Books:
101 -> Java Programming
102 -> Spring Boot
103 -> SQL Basics
Search Book ID: 102
Book Found: Spring Boot
```

Case Study 4 - Student Ranking System

Java Collections Framework — Case Study

Problem Statement

A college maintains student rankings based on marks. Students should always appear in ascending order of marks, and duplicate rankings should not be stored.

Requirements Overview

Task	Collection	What Interns Build
Student Ranking	TreeSet	Automatically sorted ranking system

Requirements

Create a Student class containing:

- Rank
- Student Name
- Marks

Perform:

- Add student records.
- Maintain ascending order of marks.
- Prevent duplicate rankings.
- Display rankings.

Appropriate Collection

TreeSet<Student>

Justification

- Automatically maintains sorted order.
- Does not allow duplicates.
- Uses Comparable for ranking logic.

Sample Input

```
Amit 85
Bhavesh 92
Rohit 78
Priya 88
```

Expected Output

Student Rankings

```
Rohit 78
Amit 85
Priya 88
Bhavesh 92
```